Compiler Arena

Uma linguagem. Um compilador. Robôs em execução.

Integrantes: [preencher]
Professor(a): [preencher]
Instituição: [preencher]

Objetivo e requisitos

- PDF: compilar, validar e executar uma linguagem de robôs.
- Semântica, IR, otimização e objeto são o centro do trabalho.
- API mínima + arena funcional + execução justa por ticks.
- Extensões: tipos, escopos, JSON, GUI, testes e relatórios.
- Sem exec(), eval() ou execução direta da AST.

Arquitetura: a transformação é real

Fonte UTF-8 > Tokens > AST > Semântica

IR tipada > Otimização > Bytecode > VM / arena

CLI e núcleo independem da interface gráfica.

RoboLang: o robô Alpha

```
robot Alpha {
    int distancia = 10 + 20;
    move(distancia + 0);
    while (energy > 0) {
        scan();
        if (enemy.visible) {
            rotate(enemy.direction);
            fire(3);
        } else {
            rotate(15);
        }
    }
}
```

Um robô por arquivo
Declaração antes do uso
Parênteses e chaves
Comandos sem retorno

SHA 03e6cdfd8fc0

Tipos, operadores e controle

- int, float finito, bool e string Unicode.
- Única promoção implícita: int para float.
- Precedência: unários, produto, soma, ordem, igualdade, &&, ||.
- if / else if / else, while, break e continue.
- Constantes, blocos léxicos e operadores compostos.

Análise léxica: caracteres → tokens

```
TIPO                LEXEMA      VALOR     POSIÇÃO
PALAVRA_RESERVADA   int         int       3:5
IDENTIFICADOR       distancia   distancia 3:9
OPERADOR            =           None      3:19
NUM_INTEIRO         10          10        3:21
OP_SOMA             +           None      3:24
NUM_INTEIRO         20          20        3:26
PONTO_VIRGULA       ;           None      3:28
```

Scanner manual • comentários preservam posições • maior operador primeiro

Parser recursivo e AST

```
DECL [distancia] @3:5
  BIN [+] @3:24
    LITERAL [10] @3:21
    LITERAL [20] @3:26
```

A forma de 10 + 20 desaparece em etapas posteriores.

Semântica: nomes, tipos e escopos

```
id: 0
name: distancia
type: int
scope: 1
mutable: True
line: 3
column: 5
constant: None
```

Tabela de símbolos real

Cada declaração tem um ID.
O backend usa esse ID.
Tipos são validados antes da IR.

Diagnóstico e bloqueio do inválido

```
fire(true);
```

Erro semântico na linha 9, coluna 13:
fire espera int entre 1 e 3; recebido bool.

Semântica: erro | IR: não executada | Bytecode: ausente

IR tipada: quádruplas e saltos

```
t1 = BIN:+ 10, 20
s0 = COPY t1
t2 = BIN:+ s0, 0
CMD:move t2
L1 = LABEL
t3 = SENSOR 'energy'
t4 = BIN:> t3, 0
L2 = JZ t4
CMD:scan
```

op / arg1 / arg2 / result + tipo + linha/coluna

IR original e otimizada: o mesmo Alpha

Original

```
t1 = BIN:+ 10, 20
s0 = COPY t1
t2 = BIN:+ s0, 0
CMD:move t2
```

Otimizada

```
s0 = COPY 30
CMD:move 30
```

Alpha completo: 21 → 19 instruções IR (9.52%).

Trecho isolado: 5 → 3 (40%). Não é medida de tempo.

Otimização: três técnicas, com limites

- Constant folding: 10 + 20 torna-se COPY 30.
- Simplificação algébrica: int x + 0 torna-se COPY x.
- Propagação de constantes dentro de blocos básicos.
- Preserva sensores, comandos, tipos e erros observáveis.
- Equivalência funcional não garante equivalência temporal.

Bytecode próprio e máquina virtual

```
00 COPY       [0, ['c', 'int', 30]]
01 CMD:move   [['c', 'int', 30]]
02 SENSOR     [1, ['c', 'string', 'energy']]
03 BIN:>      [2, ['r', 1], ['c', 'int', 0]]
04 JZ         [['r', 2], 14]
05 CMD:scan   []
06 SENSOR     [3, ['c', 'string', 'enemy.visible']]
07 JZ         [['r', 3], 12]
08 SENSOR     [4, ['c', 'string', 'enemy.direction']]
```

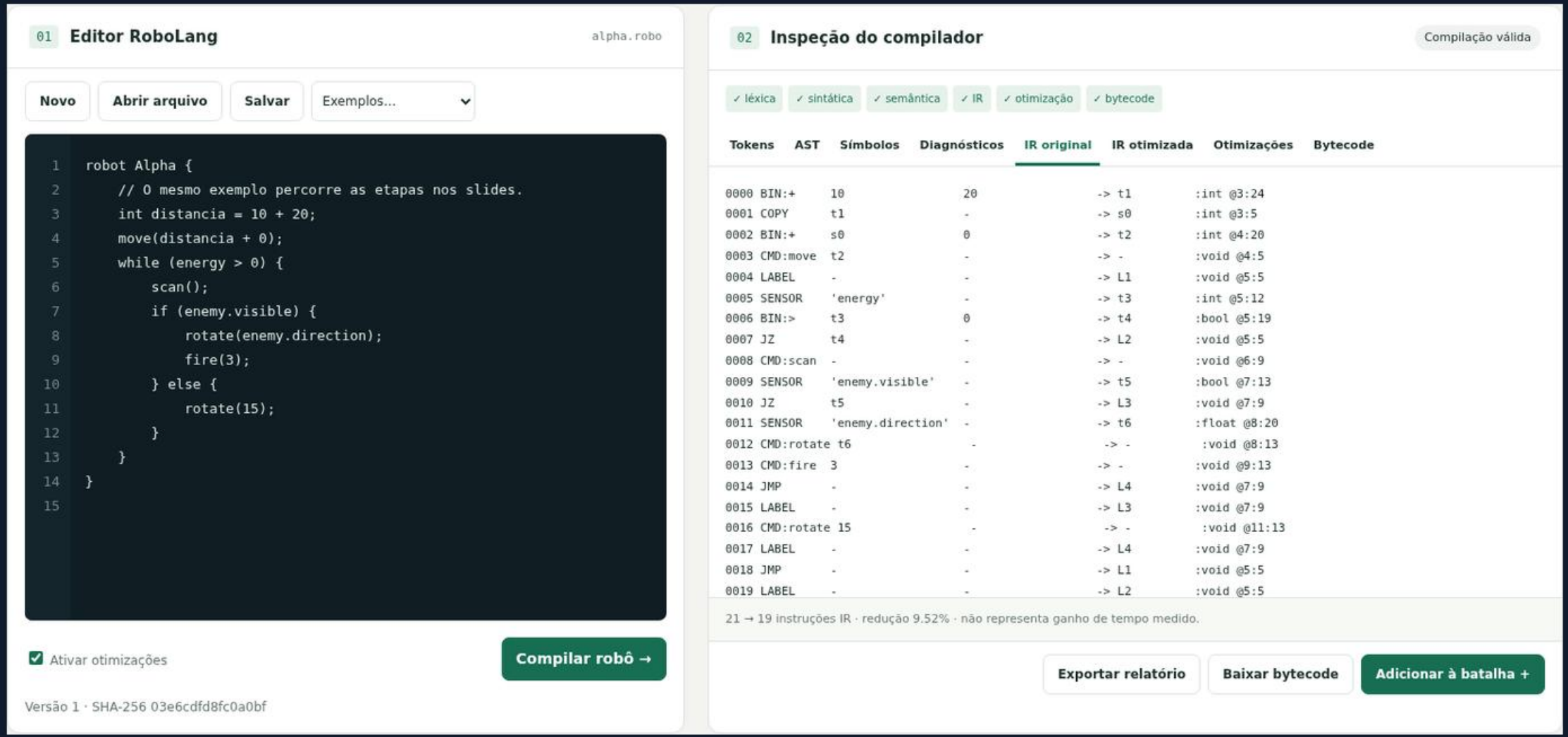
JSON validado • registradores isolados • PC persistente • sem AST

Ticks: orçamento e ações simultâneas

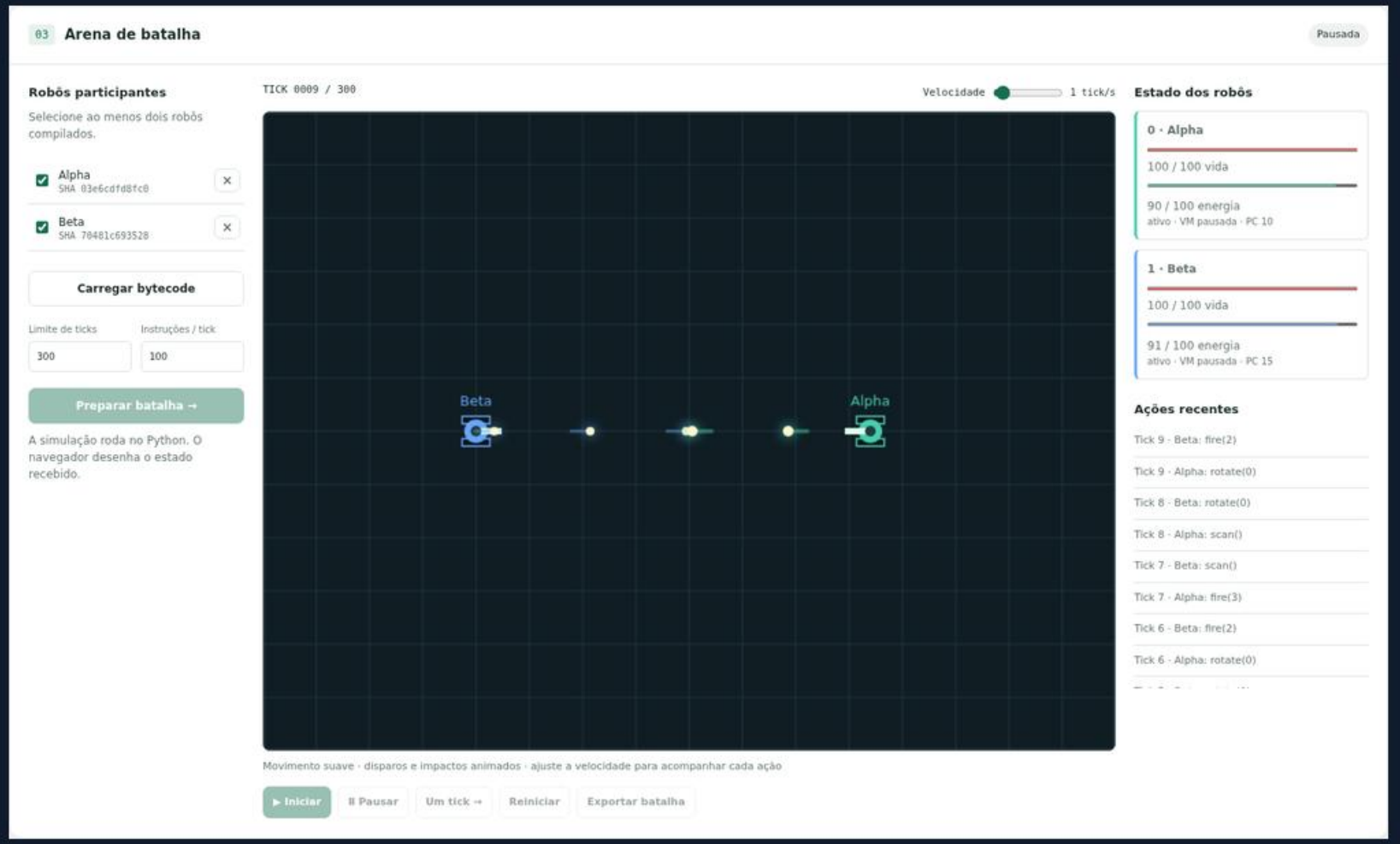
- 1 Snapshot: posições, vida, energia e último scan.
- 2 VMs: orçamento por robô; até um comando por tick.
- 3 Ações: movimento, rotação, scan e disparo.
- 4 Projéteis avançam; custos e dano simultâneos.
- 5 Novo estado; scan publicado para o próximo tick.

Loop infinito sem comando pausa ao esgotar o orçamento.

Editor: etapas reais e fonte identificada



Arena: uma única fonte de verdade



Execução de referência: Alpha venceu no tick 24.

Testes e demonstração integrada

50 testes / 0.725 s / OK

- Léxico → parser → semântica → IR → VM → arena.
- Curto-circuito, escopos, erros, orçamento e simultaneidade.
- Teste gráfico: editor, revisão do texto, HTTP, passo e reinício.
- Ao vivo: erro semântico → correção → IR → bytecode → batalha.

Limitações e evoluções

Implementado

Cadeia completa e inspecionável
CLI + editor + arena
Testes e exportação JSON
Slides gerados dos artefatos

Limites / próximos passos

Sem funções ou propagação global
Scan pode ficar desatualizado
Sem sensor de projéteis em voo
Teste web local: Chromium / Linux

A defesa deve explicar as transformações, não só a batalha.